

AI-BlogNest API - AI-Powered Blogging Backend

Project Team

Department: Computer Science

Team Leader: S. Rushyan

Team Members: K. Anandhan, M. Neelakandan, M. Sanjay, D. Jeevan Kumar

Description

AI-BlogNest API is a backend REST API for user authentication, profile access, blog management, and AI-assisted writing. It is implemented with JavaScript, Node.js, and Express.js.

MongoDB is used for data storage through Mongoose. JWT is used for authentication, and Google Gemini supports blog generation and blog summarization through dedicated API endpoints.

The confirmed project features are user registration and login, profile functionality, blog create/read/update/delete operations, blog categories, AI blog generation, and AI summarization. The API is tested locally through Thunder Client in Visual Studio Code.

This project report describes the AI-BlogNest API, its modular backend organization, the team responsible for it, and the API behavior demonstrated in the supplied test captures.

Scenario-Based Case Study

Background

The project provides a backend API for managing blog content and using AI to generate or summarize text.

Problem

- Blog content can be created, retrieved, updated, and deleted through REST API operations.
- User registration and login use JWT-based authentication.
- Blog records include a category field, as shown in the supplied API test evidence.
- The AI blog generation endpoint accepts a topic and category in the supplied test and returns generated blog content.
- The AI summarization endpoint accepts content and returns a summary in the supplied test.

Solution

The AI-BlogNest API centralizes authentication, blog operations, and Gemini-assisted generation and summarization behind REST endpoints.

Usage

- Registered user: Creates an account, logs in, and receives a JWT response from the authentication API.
- API user: Uses the profile and blog routes and can submit content to the AI generation and summarization endpoints, subject to the implemented route rules.
- API client: Sends HTTP requests and inspects JSON responses during development and testing.

Outcome

The API provides user access, blog management, AI blog generation, and AI summarization through distinct backend operations.

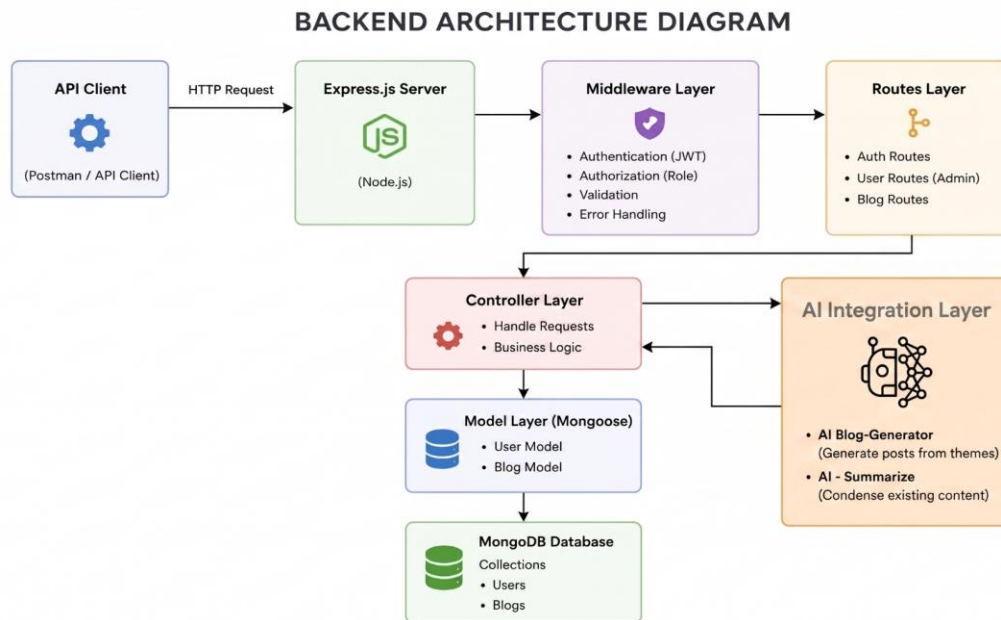
1. Software Requirements

- Operating System: Development environment shown in the supplied screenshots is Windows with Visual Studio Code.
- Node.js: JavaScript runtime used to run the Express API. The exact installed version is not specified here.
- npm: Package manager used by the Node.js project.
- **Express.js**: Lightweight routing web framework to construct backend RESTful entry points.
- MongoDB and Mongoose: Database and object modeling technology used by the project.
- Thunder Client: VS Code API client shown in the supplied request and response screenshots.
- Code Editor: Visual Studio Code, as shown in the project screenshots.

2. Hardware Requirements

- Processor: Use a computer capable of running Node.js and Visual Studio Code.
- Memory: Requirements depend on the development environment and installed tools.
- Storage: Available space is required for the project and its Node.js dependencies.

Epic:-1 PROJECT ARCHITECTURE



AI-BlogNest exposes REST endpoints through Express.js. The application uses Mongoose to access MongoDB, JWT for authentication, and Google Gemini for AI blog generation and summarization.

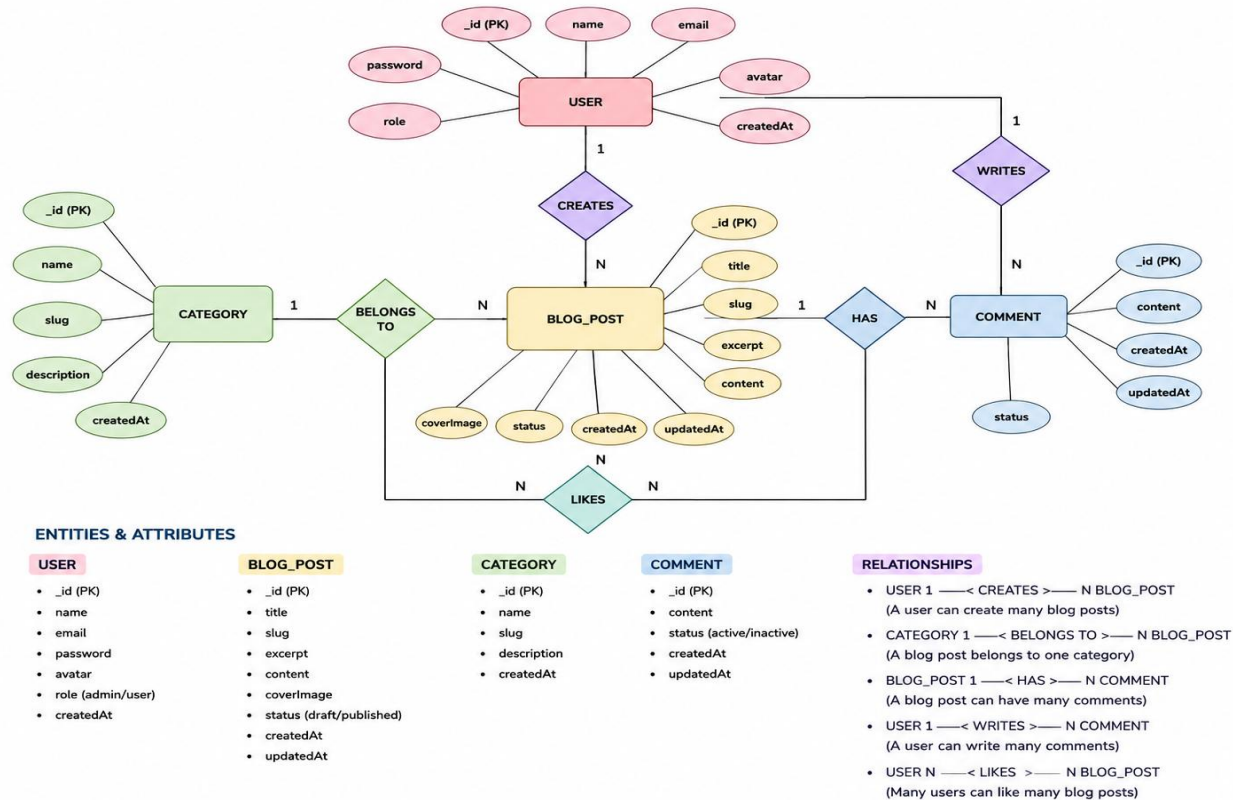
Components Definition

- **API Client:** Thunder Client in Visual Studio Code sends HTTP requests and displays JSON responses.
- **Express Server:** Receives HTTP requests and routes them to application handlers. The supplied screenshots show the local server using port 5000.
- **JWT Authentication:** The login endpoint returns a token. Protected-route behavior is determined by the implemented middleware and route configuration.
- **Application Features:** User authentication and profile functionality, blog CRUD operations, categories, AI blog generation, and AI summarization.
- **Database Layer:** Mongoose provides the application's data access layer for MongoDB.
- **AI Flow:** The known AI endpoints are POST `/api/ai/generate-blog` and POST `/api/ai/summarize`. The supplied tests show 201 Created and 200 OK responses respectively.
- **Gemini Integration:** Google Gemini is used by the API to generate blog content and summarize supplied content.

Entity-Relationship (ER) Diagram Description

User and blog records are stored through MongoDB and Mongoose. The API responses include user identity and blog content fields.

BlogNest – ER Diagram



1. User Entity

- _id (Primary Key - ObjectId)
- **name** (String, Required)
- **email** (String, Required, Unique)
- password (accepted during registration and login; not returned in the registration response)
- The registration response returns the user's ID, name, and email.

2. Blog Entity

- _id (Primary Key - ObjectId)
- author (identifier returned in the supplied create-blog response)
- title (returned in the supplied create-blog response)
- category (returned in the supplied create-blog response)

- content (returned in the supplied create-blog response)
- authorName (returned in the supplied create-blog response)
- createdAt and updatedAt (returned in the supplied create-blog response)

Key Structural Mappings

- A blog response includes an author identifier and author name.
- Blog records include a category value.
- AI blog generation and summarization are exposed through separate API routes.

Key Features

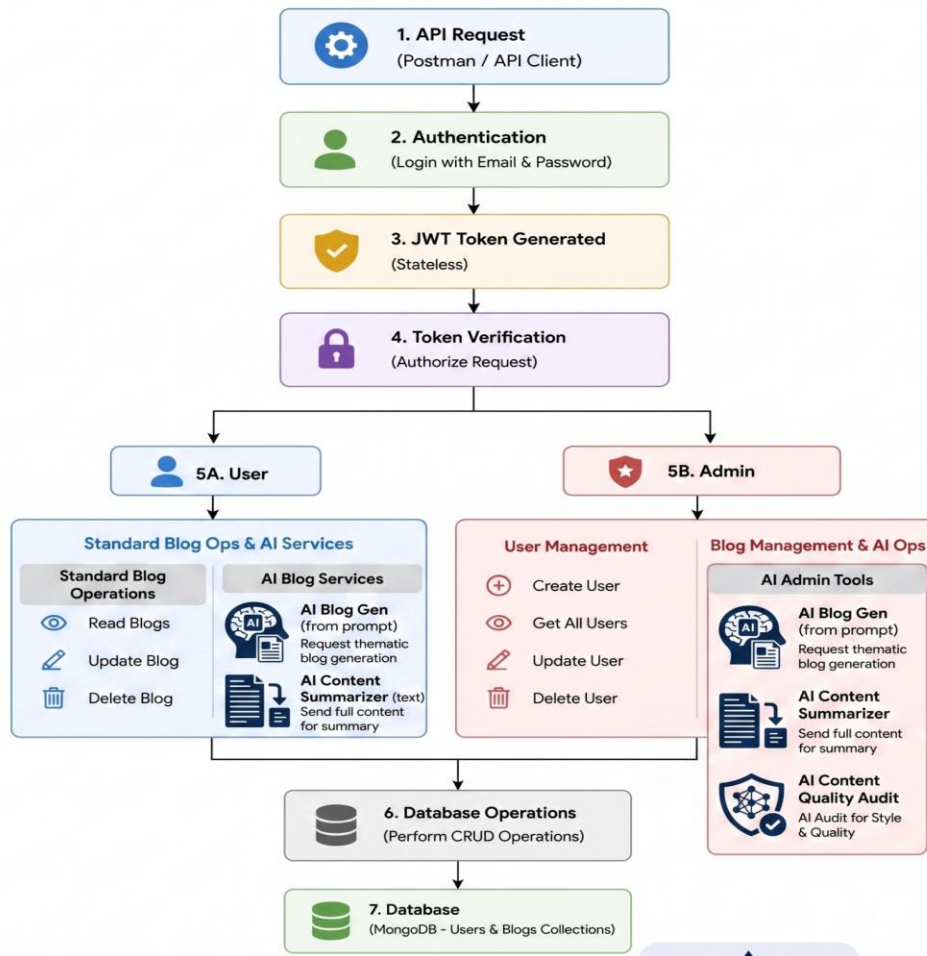
- JWT Authentication: Registration and login endpoints are shown in the supplied API testing evidence.
- Blog CRUD: Blog creation and retrieval are shown in the supplied evidence; update and delete are included in the stated project scope.
- Categories: Blog category data is included in the supplied blog and AI generation requests.
- User Profile: The supplied test log shows a successful authenticated profile request.
- API Responses: Successful requests in the supplied evidence return HTTP 200 OK or 201 Created.
- AI Blog Generation: The API uses Google Gemini to generate blog content from a topic and category in the supplied test.
- AI Summarization: The API uses Google Gemini to return a summary for supplied content.

Roles and Responsibilities

- Project Team Lead: S. Rushyan.
- Project Team Members: K. Anandhan, M. Neelakandan, M. Sanjay, and D. Jeevan Kumar.
- API User: Registers or logs in and uses the available profile, blog, and AI routes.
- The Computer Science team develops and documents the AI-BlogNest API.

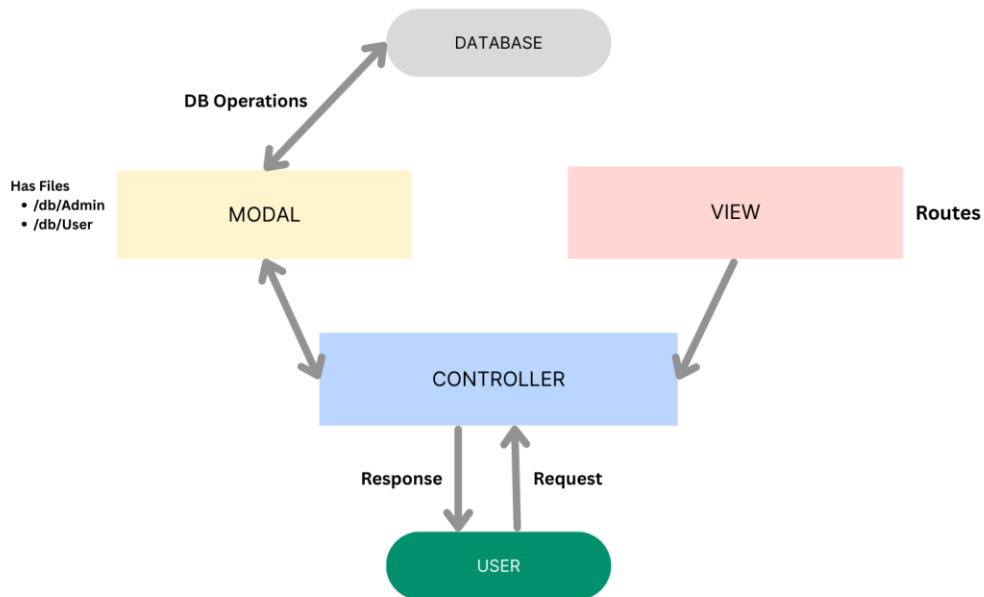
User Flow:

AI-BLOG USER FLOW DIAGRAM (Backend Only)



Modular Backend Architecture

The backend uses a modular organization with configuration, controllers, middleware, models, routes, and services under src. The supplied project structure also includes app.js and server.js.



Model Layer (Data Blueprint)

Model Layer: Mongoose models define the stored data and validation behavior in the actual project code.

Controller Layer (Execution Matrix)

Controller Layer: Controllers process incoming requests, invoke application logic, and prepare API responses.

View Layer (Routing Network Interface)

Routing Layer: Express route modules connect HTTP methods and paths to their corresponding handlers and middleware.

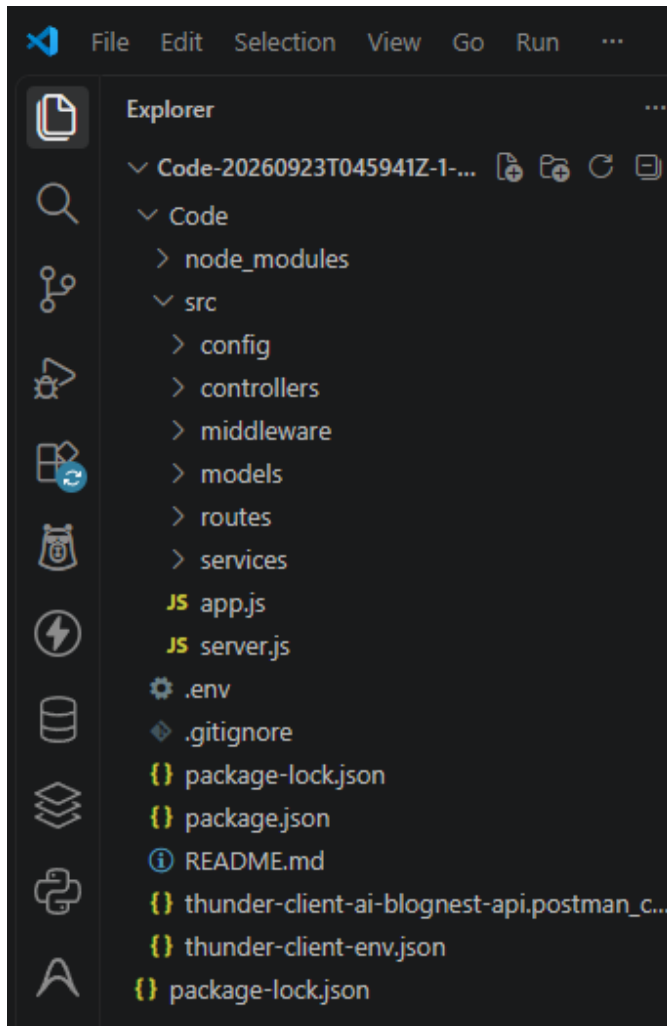
Epic2: Project Setup and Configuration

Creating project folder

1. Open the AI-BlogNest API project folder shown in the supplied folder-structure screenshot.
2. The project contains a Code folder with a src directory organized into config, controllers, middleware, models, routes, and services.

3. The Code folder also contains app.js, server.js, package.json, and project configuration files. Open the project folder in Visual Studio Code.

Project Folder Structure



Server setup (npm init)

- Open the Code project folder in the terminal integrated with Visual Studio Code.
 - `npm init -y`
- Create files:
 - `server.js` and `app.js`
- Project folders shown in the supplied structure:
 - `config`, `controllers`, `middleware`, `models`, `routes`, and `services` (inside `src`)
 - Controllers are organized under `src/controllers`.
 - Routes are organized under `src/routes`.

Epic

3:

Backend

Development

Server.js

```
JS server.js X
src > JS server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10   console.log(`Server running on port ${PORT}`);
11 });
12
```

App.js

```
JS app.js X
src > JS app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22   res.json({ message: 'AI BlogNest API is running' });
23 });
24
25 module.exports = app;
26
```

Epic4: Database Configuration

DB Connection:-

```
Explorer (Ctrl+Shift+E)
src > config > JS dbjs > ...
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    const mongoUri = process.env.MONGO_URI;
5    if (!mongoUri) {
6      console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7      process.exit(1);
8    }
9
10   try {
11     const conn = await mongoose.connect(mongoUri, {
12       useNewUrlParser: true,
13       useUnifiedTopology: true,
14     });
15     console.log(`MongoDB connected: ${conn.connection.host}`);
16   } catch (error) {
17     console.error(`MongoDB connection error: ${error.message}`);
18     process.exit(1);
19   }
20 };
21
22 module.exports = connectDB;
```

1. Configure MongoDB:

- Install Mongoose.

In your Node.js project directory, run:

`npm install mongoose`

- The supplied terminal screenshots show MongoDB connected and the server listening on port 5000. Keep database credentials in environment configuration and never publish them in the report.
- The supplied screenshots show User data returned by authentication routes and Blog data returned by blog routes. Refer to the project model files for the complete schemas.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

User Model

Mongoose schemas define the stored fields and validation rules used by the API.

```
JS User.js  X
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, 'Name is required'],
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: [true, 'Email is required'],
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: [true, 'Password is required'],
20       minlength: 6,
21     },
22   },
23   {
24     timestamps: true,
25   }
26 );
27
28 module.exports = mongoose.model('User', userSchema);
29
```

Start the backend using the command configured in the project's package.json.

Use Thunder Client in Visual Studio Code to send API requests, as shown in the supplied screenshots.

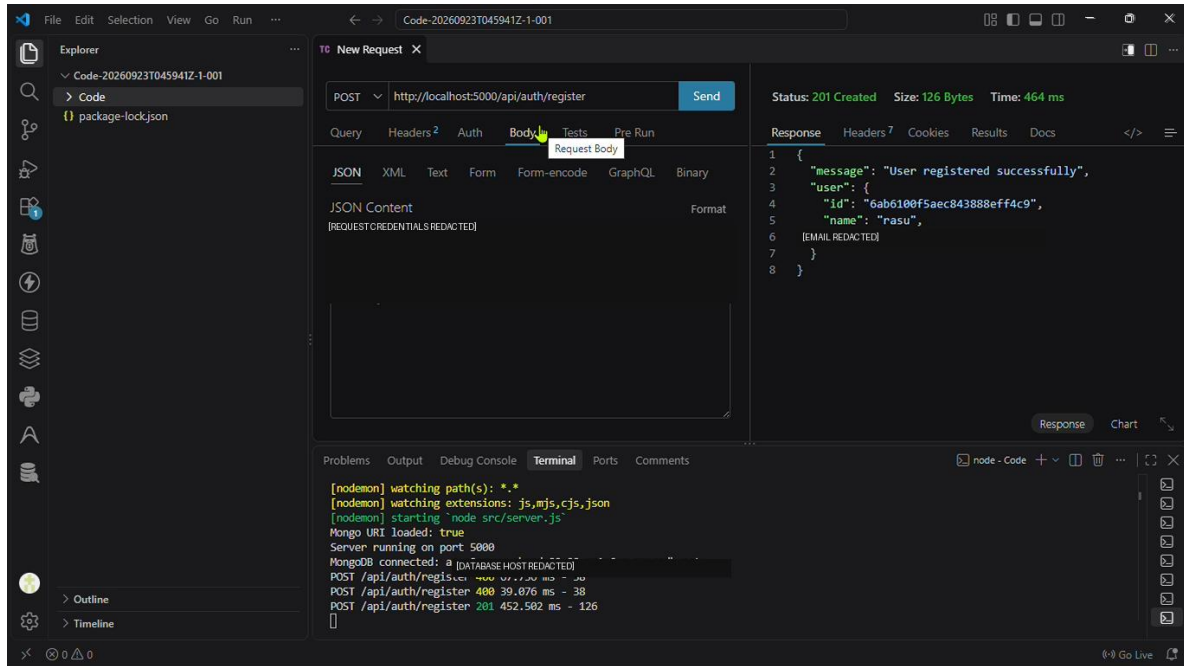
The supplied terminal screenshot shows the server running locally on port 5000.

"npm run dev"

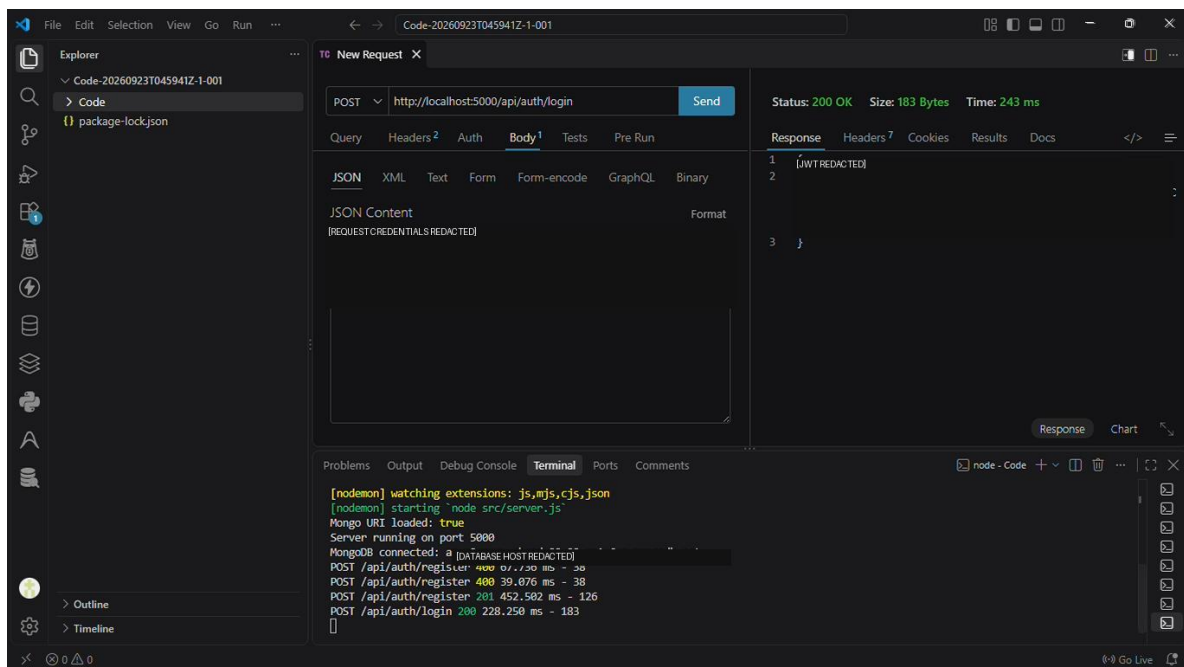
AI-BlogNest API Evidence

The following figures show the supplied API request and response results.

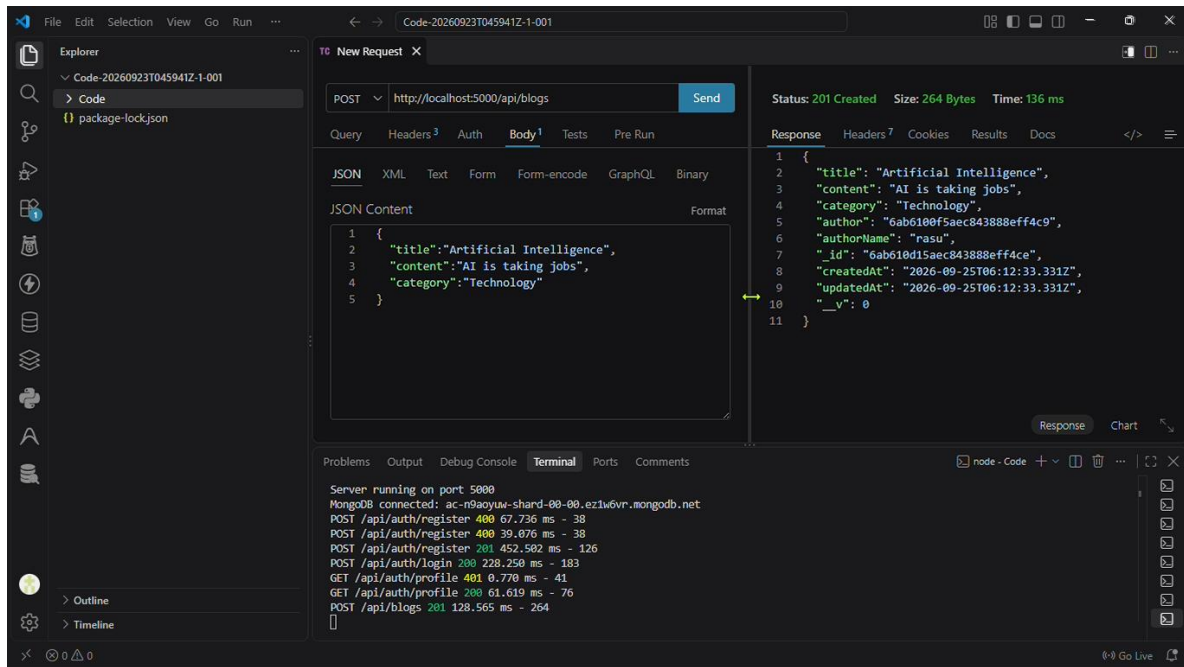
User Registration API - POST /api/auth/register - 201 Created



User Login API - POST /api/auth/login - 200 OK (credentials and token hidden)



Create Blog API - POST /api/blogs - 201 Created



The screenshot shows the VS Code interface with a REST client request to `POST http://localhost:5000/api/blogs`. The request body is a JSON object:

```
1 {
2   "title": "Artificial Intelligence",
3   "content": "AI is taking jobs",
4   "category": "Technology"
5 }
```

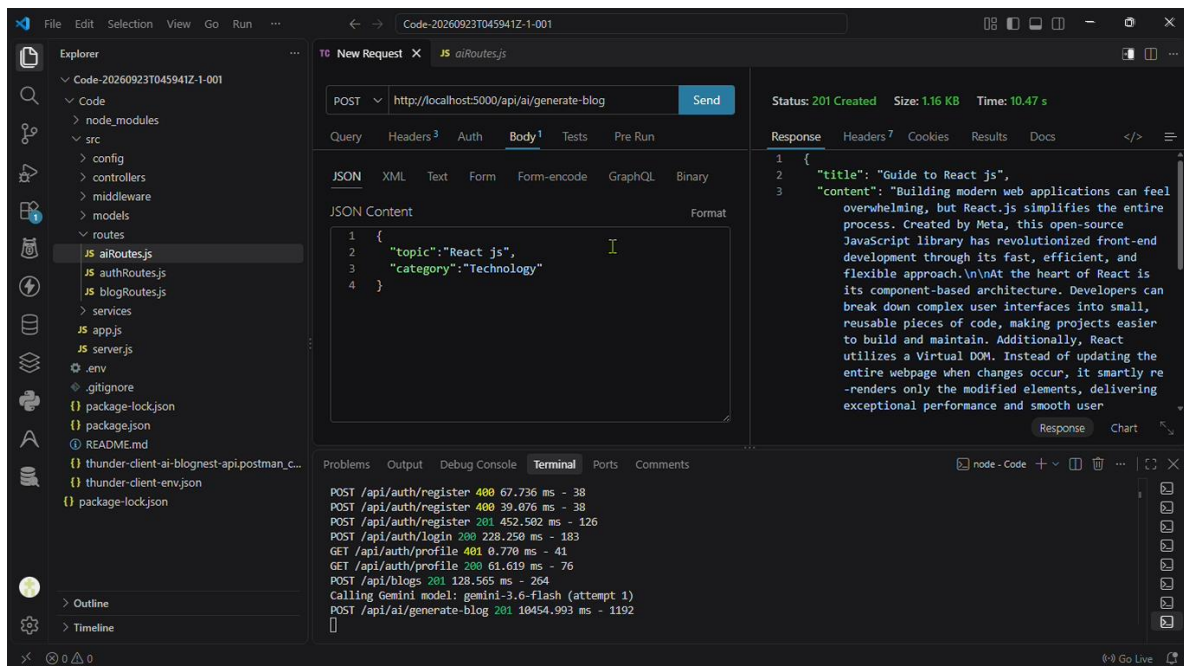
The response is a 201 Created status with a JSON object:

```
1 {
2   "title": "Artificial Intelligence",
3   "content": "AI is taking jobs",
4   "category": "Technology",
5   "author": "6ab6100f5aec84388eff4c9",
6   "authorName": "rasu",
7   "_id": "6ab610d15aec84388eff4ce",
8   "createdAt": "2026-09-25T06:12:33.331Z",
9   "updatedAt": "2026-09-25T06:12:33.331Z",
10  "_v": 0
11 }
```

The terminal shows the server running on port 5000 and the MongoDB connection. The log output includes the following messages:

```
Server running on port 5000
MongoDB connected: ac-n9ayuw-shard-00-00.ez1w6vr.mongodb.net
POST /api/auth/register 400 67.736 ms - 38
POST /api/auth/register 400 39.076 ms - 38
POST /api/auth/register 201 452.502 ms - 126
POST /api/auth/login 200 228.250 ms - 183
GET /api/auth/profile 401 0.770 ms - 41
GET /api/auth/profile 200 61.619 ms - 76
POST /api/blogs 201 128.565 ms - 264
```

Generate AI Blog - POST /api/ai/generate-blog - 201 Created



The screenshot shows the VS Code interface with a REST client request to `POST http://localhost:5000/api/ai/generate-blog`. The request body is a JSON object:

```
1 {
2   "topic": "React js",
3   "category": "Technology"
4 }
```

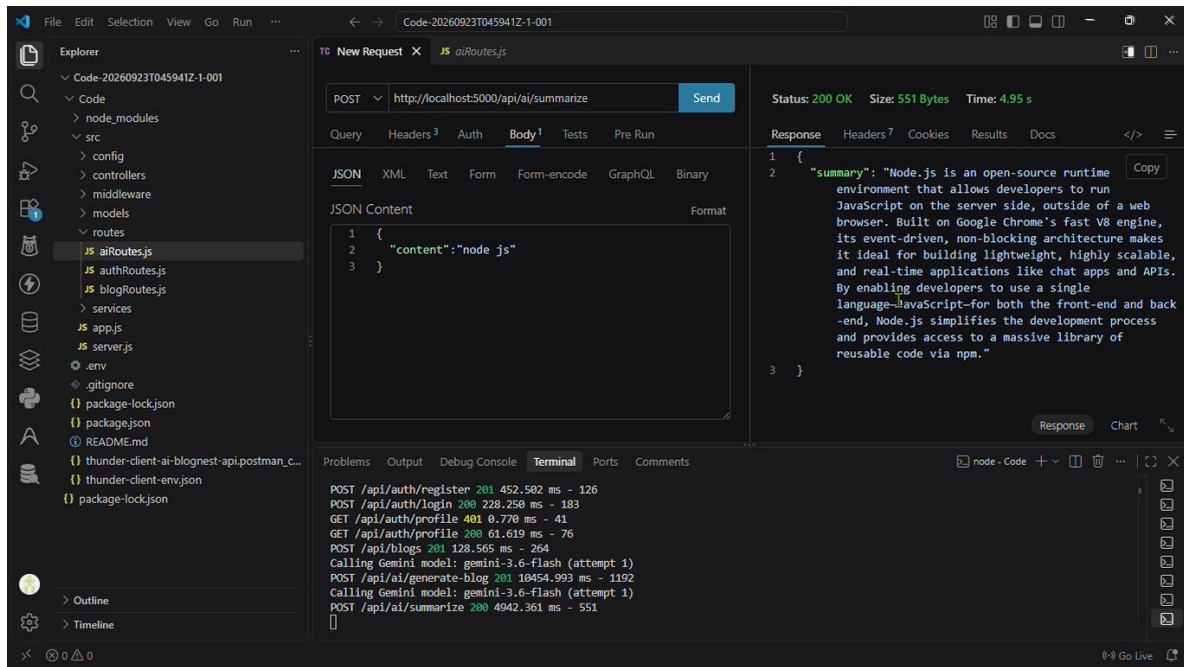
The response is a 201 Created status with a JSON object:

```
1 {
2   "title": "Guide to React js",
3   "content": "Building modern web applications can feel overwhelming, but React.js simplifies the entire process. Created by Meta, this open-source JavaScript library has revolutionized front-end development through its fast, efficient, and flexible approach.\n\nAt the heart of React is its component-based architecture. Developers can break down complex user interfaces into small, reusable pieces of code, making projects easier to build and maintain. Additionally, React utilizes a Virtual DOM. Instead of updating the entire webpage when changes occur, it smartly re-renders only the modified elements, delivering exceptional performance and smooth user
```

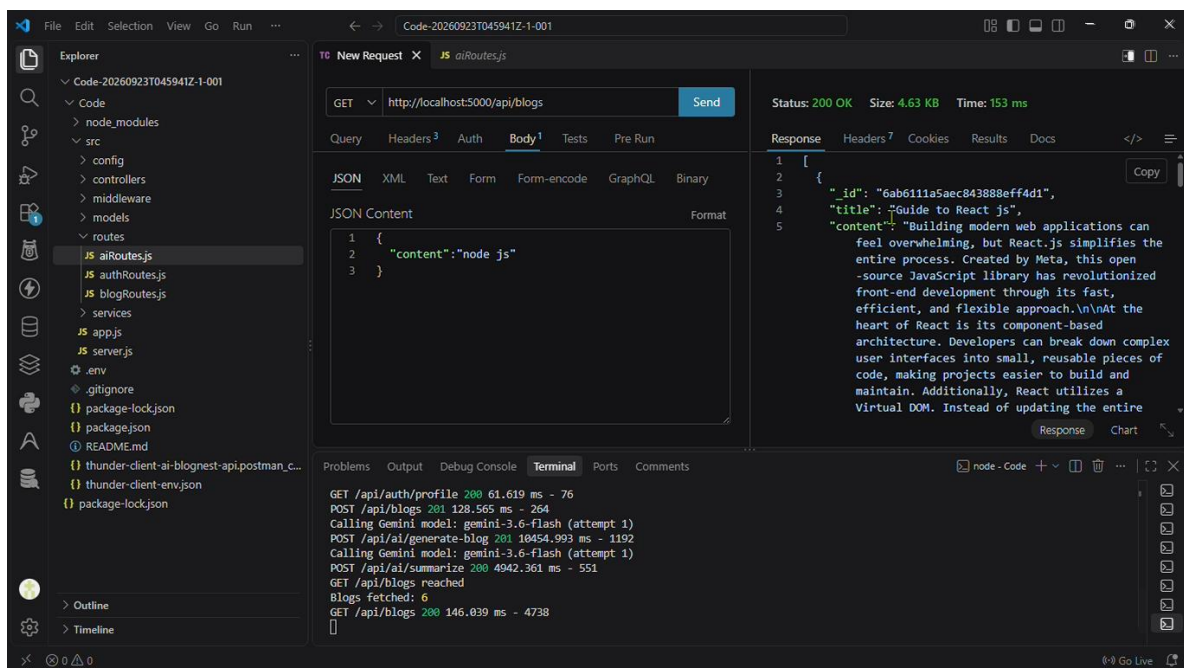
The terminal shows the server running on port 5000 and the MongoDB connection. The log output includes the following messages:

```
POST /api/auth/register 400 67.736 ms - 38
POST /api/auth/register 400 39.076 ms - 38
POST /api/auth/register 201 452.502 ms - 126
POST /api/auth/login 200 228.250 ms - 183
GET /api/auth/profile 401 0.770 ms - 41
GET /api/auth/profile 200 61.619 ms - 76
POST /api/blogs 201 128.565 ms - 264
Calling Gemini model: gemini-3.6-Flash (attempt 1)
POST /api/ai/generate-blog 201 10454.993 ms - 1192
```

Summarize Blog - POST /api/ai/summarize - 200 OK



Fetch Blogs - GET /api/blogs - 200 OK



Demo video: [Demo video](#)

API testing: [API testing video](#)

Code: [Code](#)